UWE Bristol | University of the West of England

Lecture 1: Introduction

Week 1

Advanced Software Development

UFCF8S-30-2

What will we cover today?

- Module aim and objectives
- Overview of software methodologies
- Software complexity and why modelling is needed

General Information

- Prior knowledge would be helpful:
  - Problem solving, software development, programming
  - Principles of programming (high-level programming e.g., Python/Java/C++)
  - Web development and DB design (e.g., Python/MySQL or other DBs)

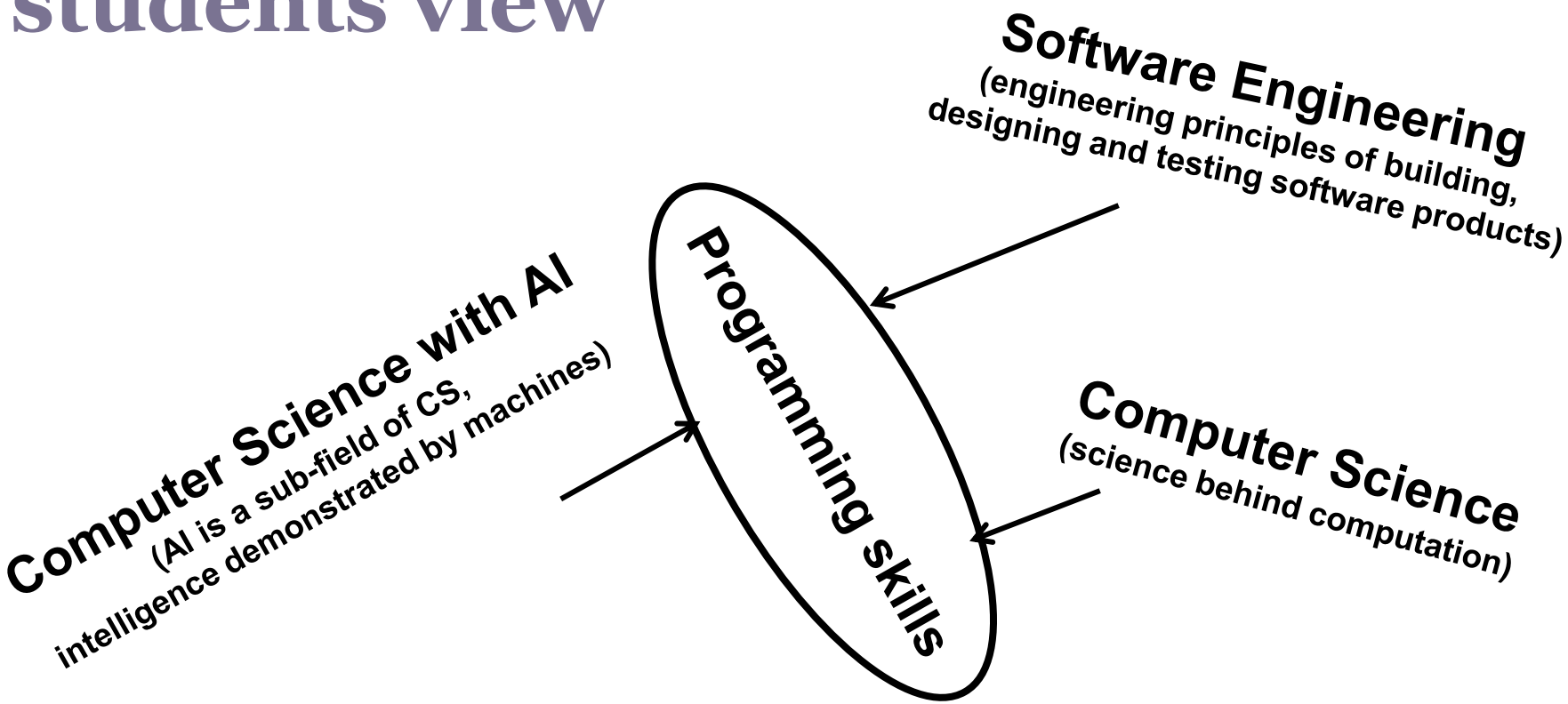
Module aims

This module aims to apply contemporary software development approaches in order to develop advanced software applications.

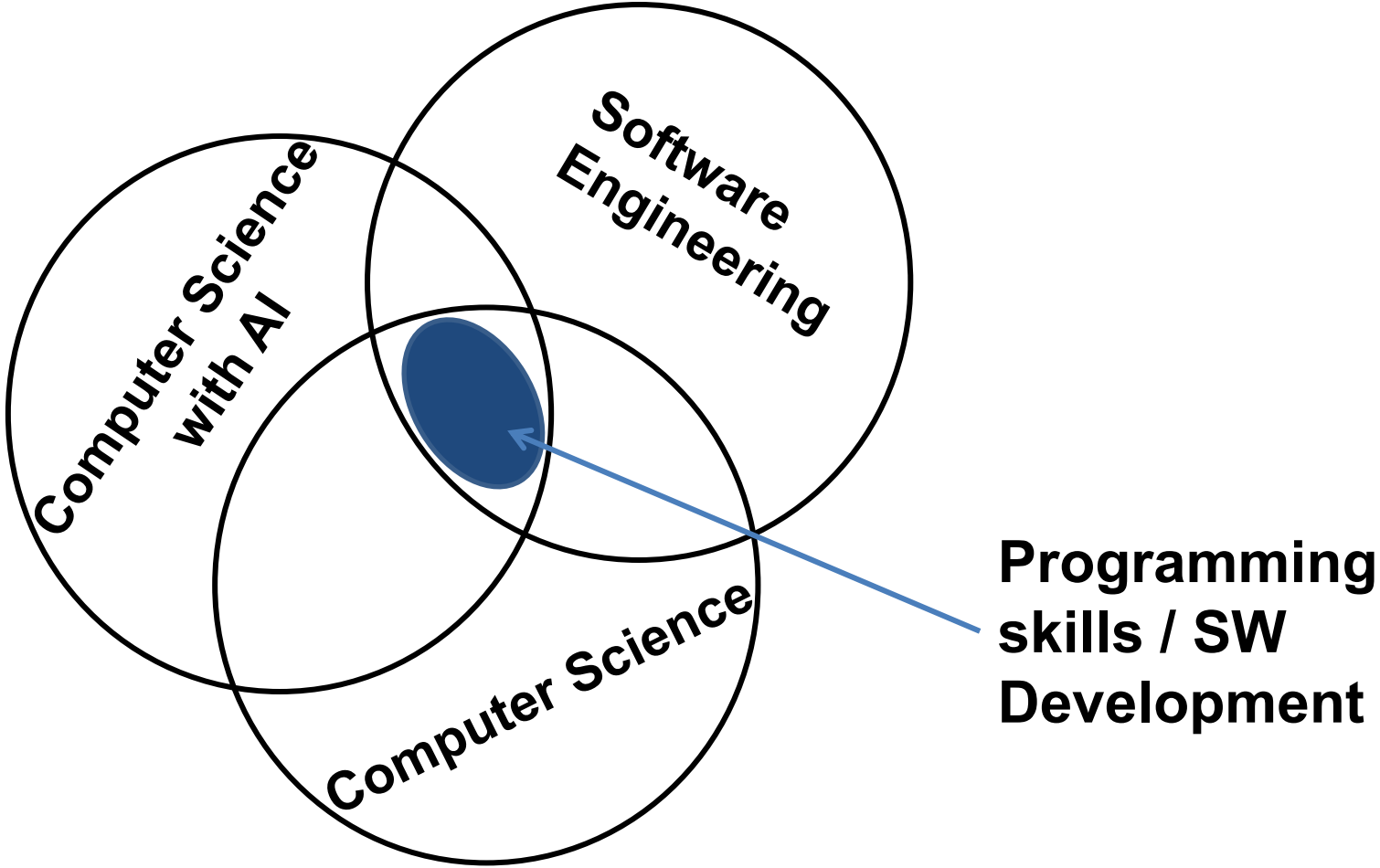
More specifically

- to convey a basic understanding of the concepts, problems, and techniques of software systems
- to show how these can be used to design and implement reasonably large size software systems, while working in a team (to a professional standard)
- to develop new problem-solving skills

This is how industry/parents /students view



This is the reality of the situation



Module objectives

- Understanding user conceptual models and development of better specification;
- Learn the basic methodologies, traditional software development, and Agile methods;
- Understanding the concepts of faulty design and the principles to follow (SOLID);
- Gain knowledge of the practice of constructing large systems, including concurrent systems and distributed systems;
- Design and implement Graphical User Interfaces;
- Gain knowledge on systematic approach to the design, development, testing, maintenance of software;
- Reusability of code and design (Design pattern)

Internet/reading Information

UWE Bristol University of the West of England

- **Readling List** – Check Blackboard for the reading list
- Hunt, J. (2019) Beginner's Guide to Python 3 Programming, Springer
- Hunt, J. (2019) Advanced Guide to Python 3 Programming, Springer
- Fowler, F (2004) UML Distilled: A Brief Guide to the Standard Object Modeling Language
- Miles, R. and Hamilton, K. (2006) Learning UML 2.0, O'Reilly
- Pooley, R, Stevens, P. (1999) Using UML Software Engineering with Objects and Components, Addison Wesley
- McLaughlin, B., Pollice, G. and West, D. (2007) Head First Object-oriented Analysis and Design, O'Reilly
- Booch (1994) Object-oriented Analysis and Design with Applications, 3rd edition, Addison Wesley
- Bernd Bruegge, Allen Dutoit: "Object-Oriented Software Engineering: Using UML, Patterns, and Java", 2010, Prentice Hall

All labs and coursework will be in **UML and Python**. Learning materials will be available on Blackboard.

Assessment

- Group Project assignment (100% of module Overall)
- Portfolio (75%)
  - Design, development, and testing

- Viva/Demo (25%)
  - During Practical sessions

Groupwork

- Groupwork - an essential element in modern software development to achieve common objectives but it can be challenging: diversity - different skillset, viewpoints, priorities/commitments, etc.

- Have a clear understanding about tasks and responsibilities
- Be accommodating - compromise, negotiate etc.
- Focus on strengths of individuals – special interest and respect diversity
- Plan ahead – structure it, make a routine, get support
- Start early – don't leave it to a day before submission deadline!
- Learn from each other and share knowledge
- Manage project: Plan, coordinate/talk and implement – monitor progress e.g., logs
- Do self/group assessment or reflection
- Define roles –
  - co-ordinator, specialist, etc.
  - see possible roles descriptors - https://www.belbin.com/about/belbin-team-roles

Groupwork

- Develop Team Charter/Contract as early as possible
- Attend all scheduled group meetings
- Communicate clearly and in a timely manner with other group members
- Respect and involve other team members, listening to their ideas in a non-judgmental way
- Give constructive feedback to other group members
- Respond appropriately to constructive feedback from other group members
- Agree appropriate tasks and deadlines within the group
- Meet internal deadlines agreed within the group
- Apply creative problem solving
- Demonstrate co-operative and professional behaviour
- Contribute equally to the group assignment
- Be honest, open and objective within peer assessment.
- Seek help as early as possible and before final deadlines if disputes or problems arise.

Remember failure on one (or more) group member's part can severely effect achieving project objectives as well as can bring bad repute to software development team/organisation!

Improving Performance

- To transfer knowledge/information from your short-term memory to your long-term memory, that information must be imposed on your mind at least three times
- **3Ps:** Look at the slides or reading material before class (Prepare), Attend lectures/practicals and engage in discussions (**Participate**), **Review/Practise** the lecture evening/night (self-study)
- In addition to this, you should: Get a reasonable night's sleep, Eat a healthy breakfast
- Most important- try your best to attend all the lectures/labs

Flexibility

- This is your class, I am happy to go any way you like as long as we learn the “ASD”
- Take responsibility for your own learning
- We’ll learn from each other.
- We are facilitator (and not just teachers ☺)

Lets move on…

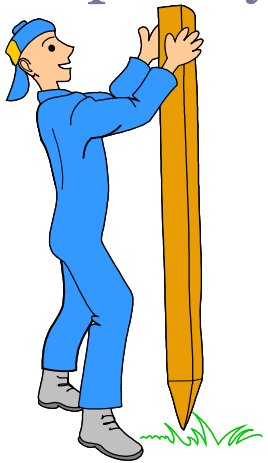
Developing software systems…complex!

UWE Bristol University of the West of England

- Complex ≠ complicated
- Complex = composed of many simple parts related to one another
- Complicated = not well understood, or explained

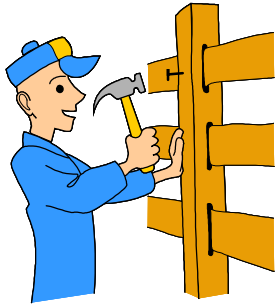
Scheduling Fence Construction Tasks:

Complexity Example

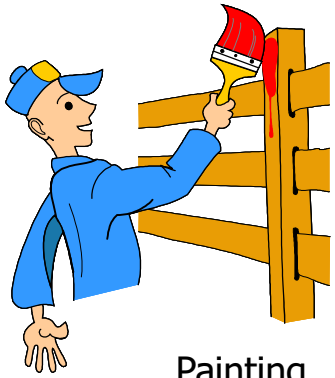


Setting posts
[3 time units]

Cutting wood
[2 time units]



Nailing
[2 time units for unpainted;
3 time units otherwise]



Painting
[5 time units for uncut wood;
4 time units otherwise]

…shortest possible completion time = ?

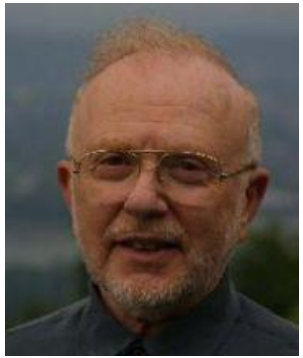
[⇒ "simple" problem, but hard to solve without a pen and paper]

Another complex problem example…

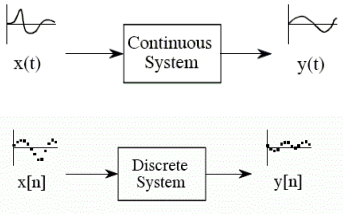
Can you reflect on complexity of a booking system? E.g., hotel booking, travel booking, cinema ticket booking etc.

Why is software development difficult?

David Lorge Parnas is an early pioneer in software engineering who developed the concepts of modularity and information hiding in systems which are the foundation of object oriented methodologies.



- The problem domain (also called application domain) is difficult
- The solution domain is difficult
- The development process is difficult to manage
- Software offers extreme flexibility
- Software is a discrete system
  - Continuous systems have no hidden surprises
  - Discrete systems can have hidden surprises! (Parnas)



Software Engineering: A Problem Solving Activity

- **Analysis**: Understand the nature of the problem and break the problem into pieces
- **Synthesis**: Put the pieces together into a large structure

For problem solving we use

- **Techniques (methods):**
  - Formal procedures for producing results using some well-defined notation
- **Methodologies:**
  - Collection of techniques applied across software development and unified by a philosophical approach
- **Tools:**
  - Instrument or automated systems to accomplish a technique

Factors affecting the quality of a software system

- **Complexity**:
  - The system is so complex that no single programmer can understand it anymore
  - The introduction of one bug fix causes another bug

- **Change**:
  - The "Entropy" of a software system increases with each change: Each implemented change erodes the structure of the system which makes the next change even more expensive ("Second Law of Software Dynamics").
  - As time goes on, the cost to implement a change will be too high, and the system will then be unable to support its intended task. This is true of all systems, independent of their application domain or technological base.

Dealing with Complexity….

1. Abstraction
2. Decomposition
3. Hierarchy

Models are used to provide abstractions

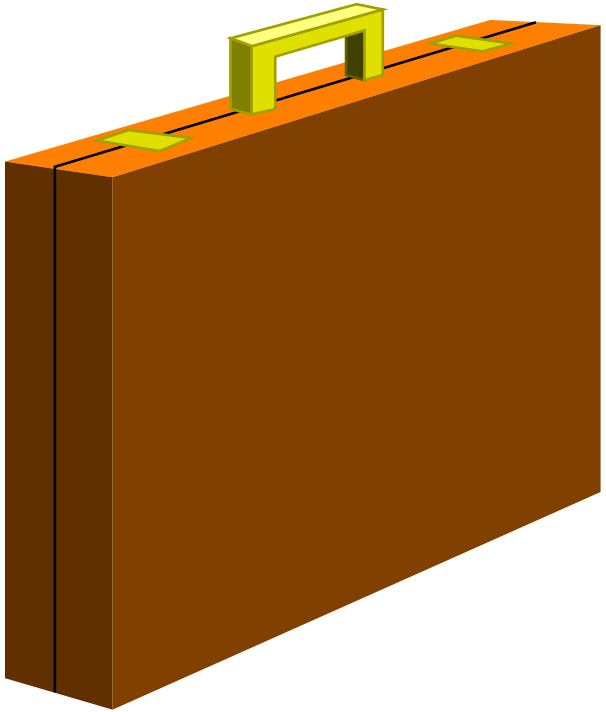
- In general, a model is a simplified and generalised version of reality, from which the unessential details have been removed
- A model is an abstraction of a system and is used to describe a specific perspective of the system

A system that no longer exists

An existing system

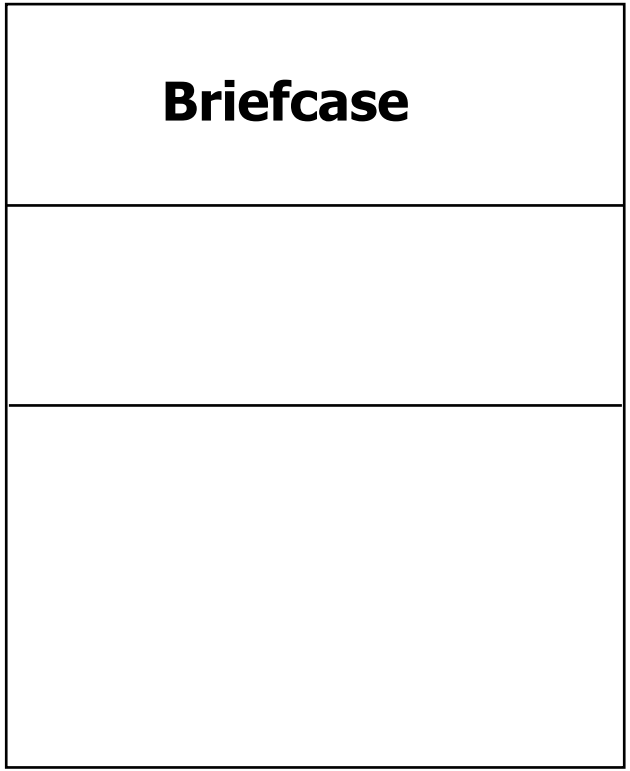
A future system to be built

What is this Thing?



Modeling a Briefcase

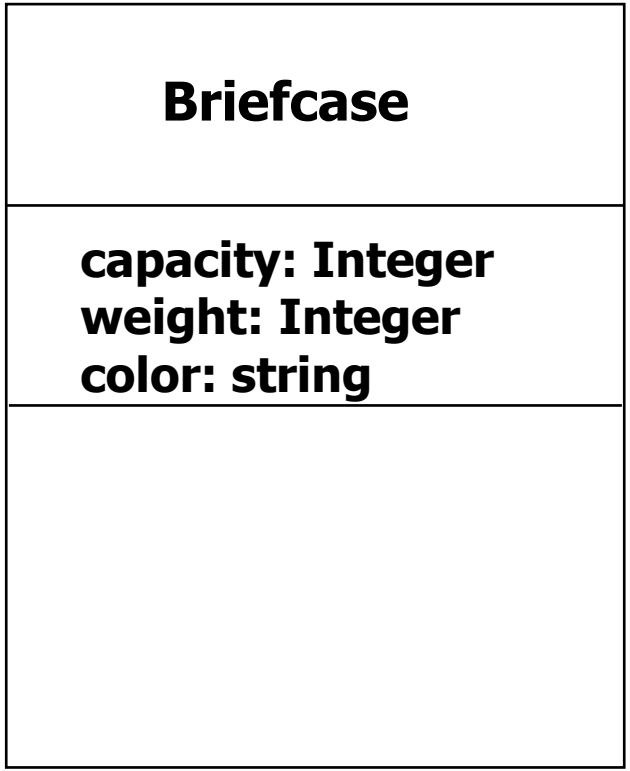
How do we represent a briefcase in computer?



Modeling a Briefcase…

What can we do with a briefcase?

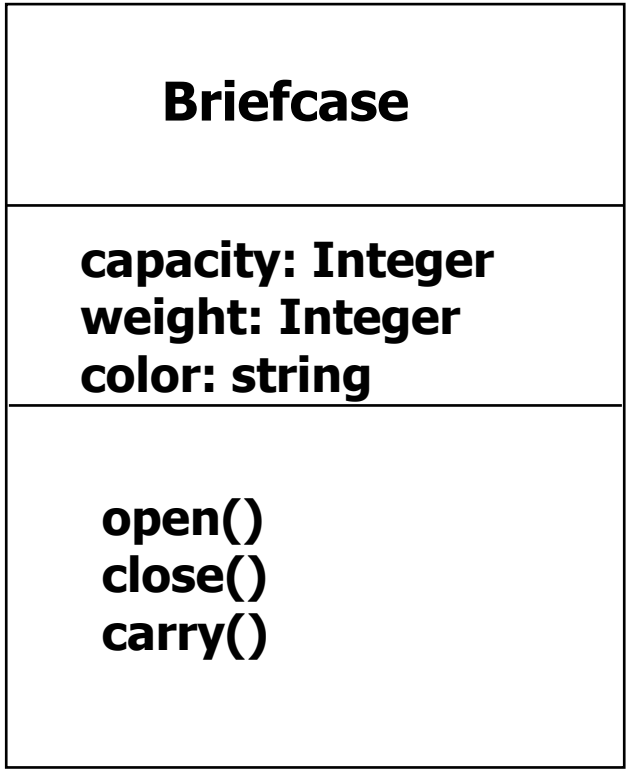
(What does it do?/ How does it behave?)



Modeling a Briefcase…

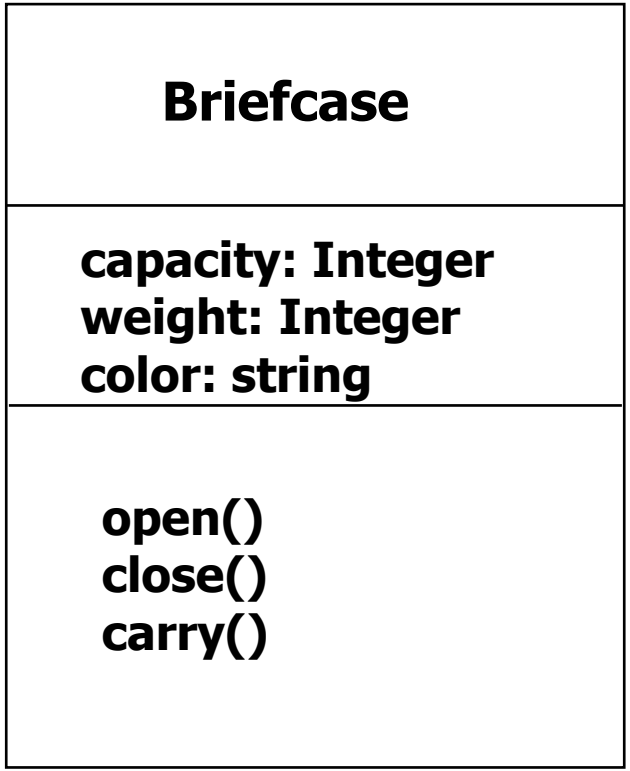
What can we do with a briefcase?

(What does it do?/ How does it behave?)

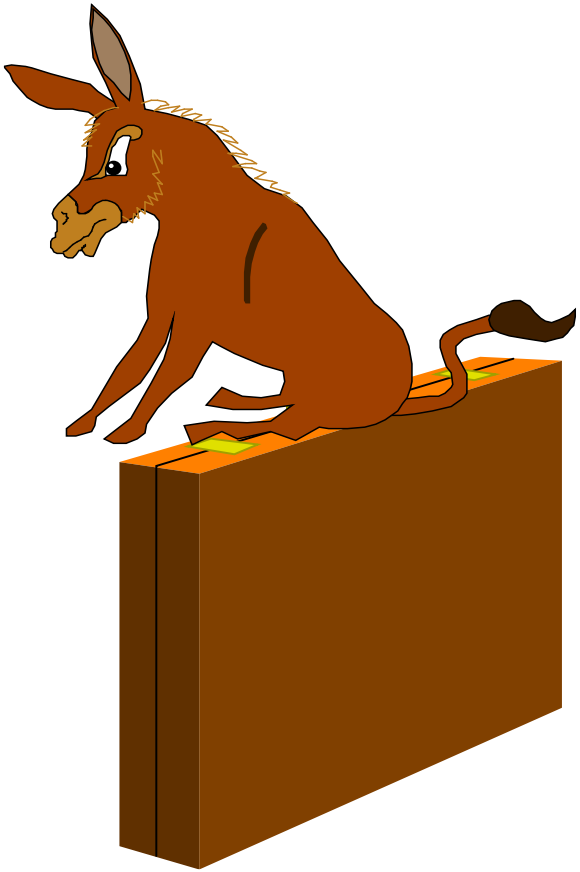


Modeling a Briefcase…

what else can we do with it?



Modeling a Briefcase…



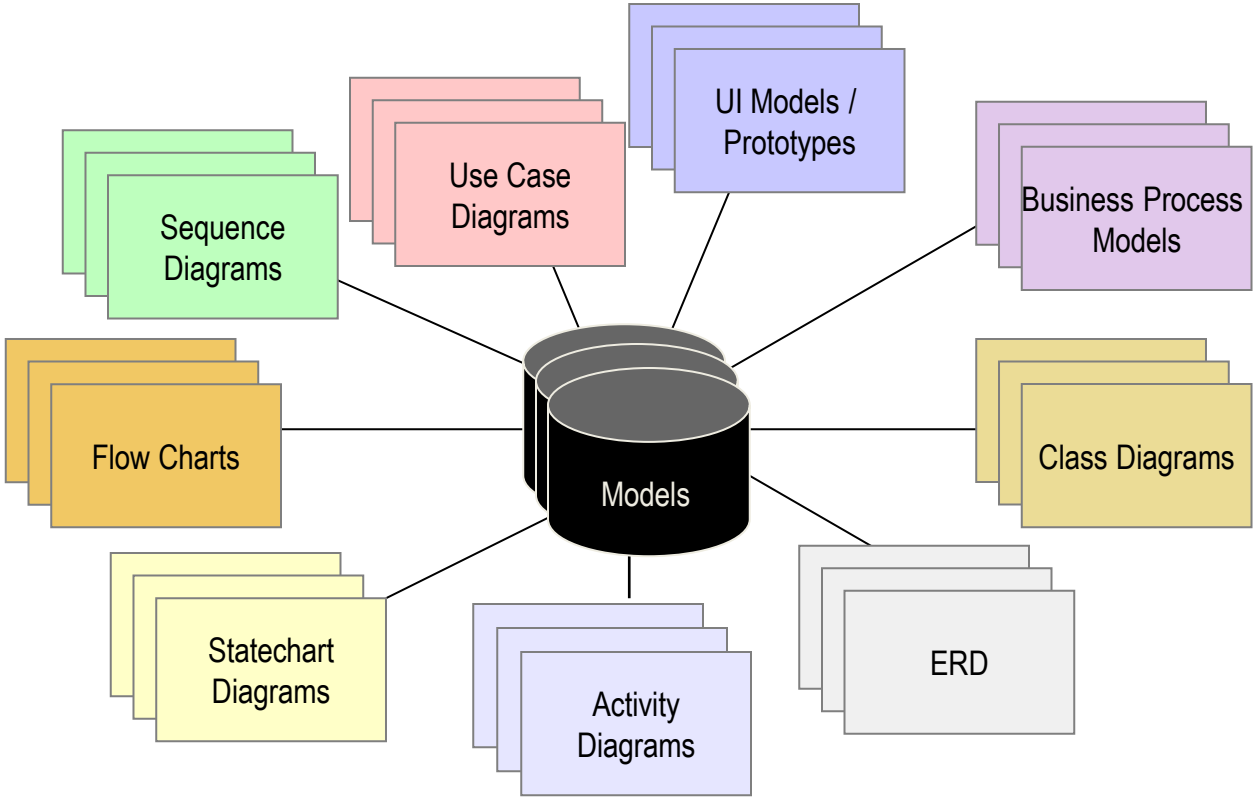
Briefcase
capacity: Integer weight: Integer color: string
open() close() carry() sitOnIt()

Observations/Questions

UWE Bristol University of the West of England

- Why did we model the thing as “Briefcase”?
- Why did we not model it as a chair?
- What do we do if the *sitOnIt()* operation is the most frequently used operation?
- The briefcase is only used for sitting on it. It is never opened nor closed.
  Is it a “Chair”or a “Briefcase”?
- How long shall we live with our modeling mistake?
- Who can help us answer these questions?

Various Types of Model

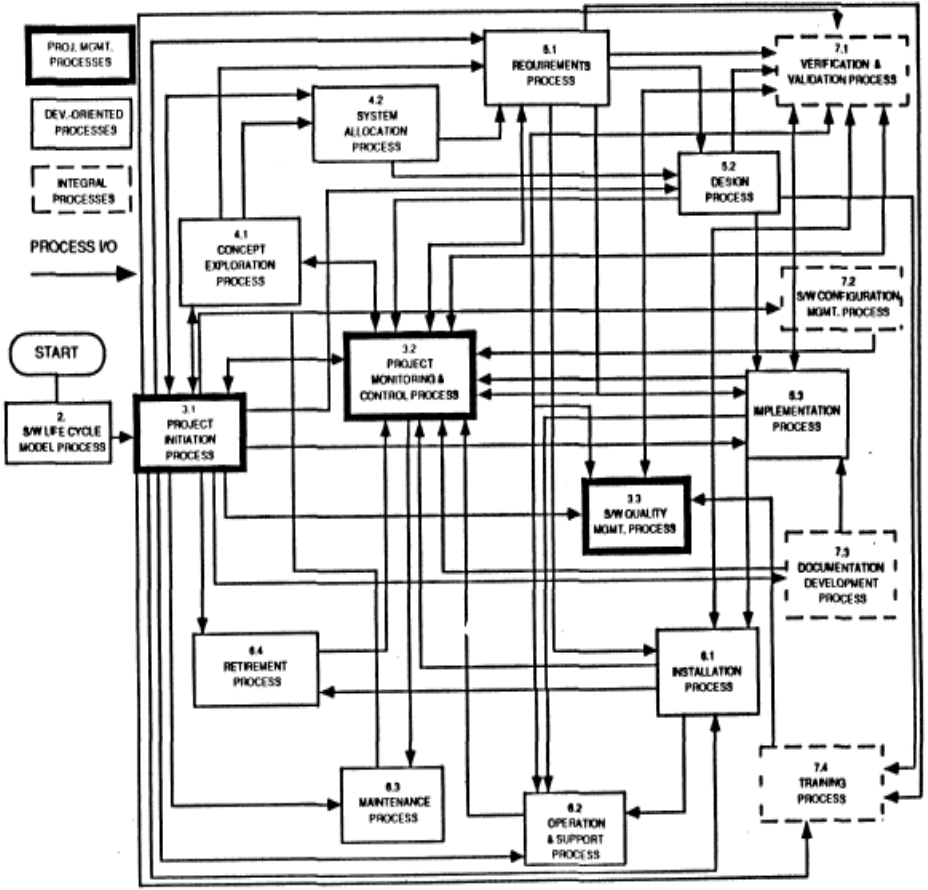


Adapted from J Hunt – CTaP

Dealing with Complexity....

1. Abstraction
2. **Decomposition**
3. **Hierarchy**

What is this?



Exercise to see brain capacity to remember/process things

Utter these words and then try to repeat

fairground
ungeared
rambam
doomsday
fogging
gazehound
antiphysical

lonelier
glumpier
cicatricose
californian
levelled
sight
forswearer

keister
charterable
themistocles
irreplevisable
minuting
boliviano

Decomposition..

- Complex systems are hard to understand
  - The 7 +- 2 phenomena
  - “The Magical Number Seven, Plus or Minus Two: Some Limits on Our Capacity for Processing Information”
  - “It is often interpreted to argue that the number of objects an average human can hold in working memory is 7 ± 2. This is frequently referred to as Miller's Law”

Decomposition…

- Complex systems are hard to understand
  - The 7 +- 2 phenomena
    - Our short term memory cannot store more than 7+-2 pieces at the same time -> limitation of the brain
    - **UWE Phone Number: 44 117 32 87216**
- Chunking:
    - *Group collection of objects to reduce complexity*
    - **Country-code**, **city-code**, **UWE-code**, **Office-Part**

Decomposition…

- Chunking: decompose into smaller parts
  - Group collection of objects to reduce complexity
  - Country-code, city-code, UWE-code, Office-Part

UWE Phone Number

Country-Code | City-Code | UWE-code | Office-Part

Decomposition…

– This leads us to classes and objects

"Chunks"

UWE Phone Number

Country-Code

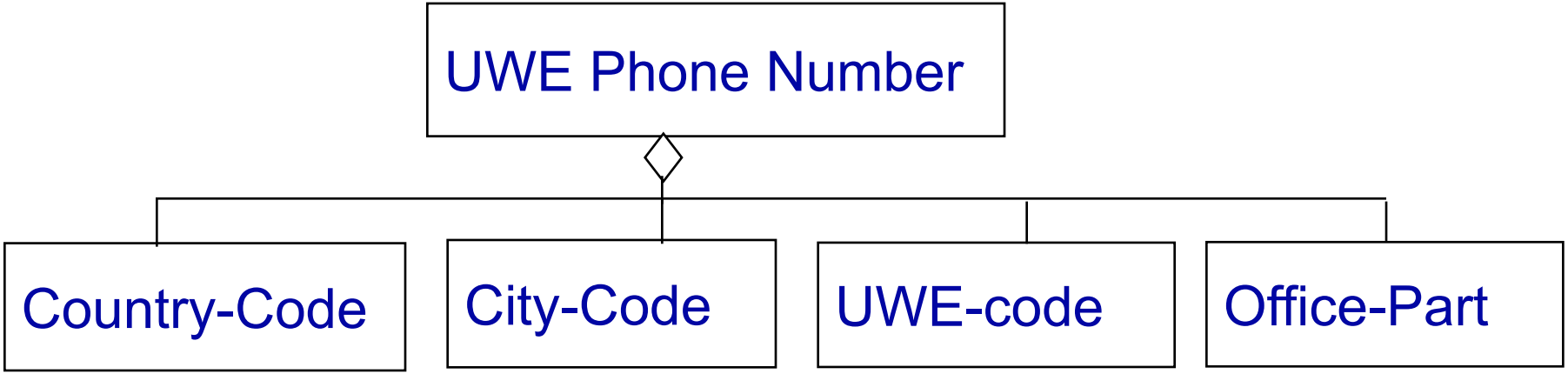
City-Code

UWE-code

Office-Part

Hierarchy…

- Another way to deal with complexity is to provide relationships between these "chunks"
- One of the most important relationships is hierarchy
- Two special hierarchies
  - "Part-of" hierarchy
  - "Is-kind-of" hierarchy (More detail in the next lectures…)



So where are we right now?

UWE Bristol University of the West of England

- Three ways to deal with complexity:
  - Abstraction
  - Decomposition
  - Hierarchy
- Object-oriented decomposition is a good methodology
  - Unfortunately, depending on the purpose of the system, different objects can be found
- How can we do it right?
  - Many different possibilities
  - Our current approach: Start with a description of the functionality (Use case model), then proceed to the object model
  - This leads us to the software lifecycle

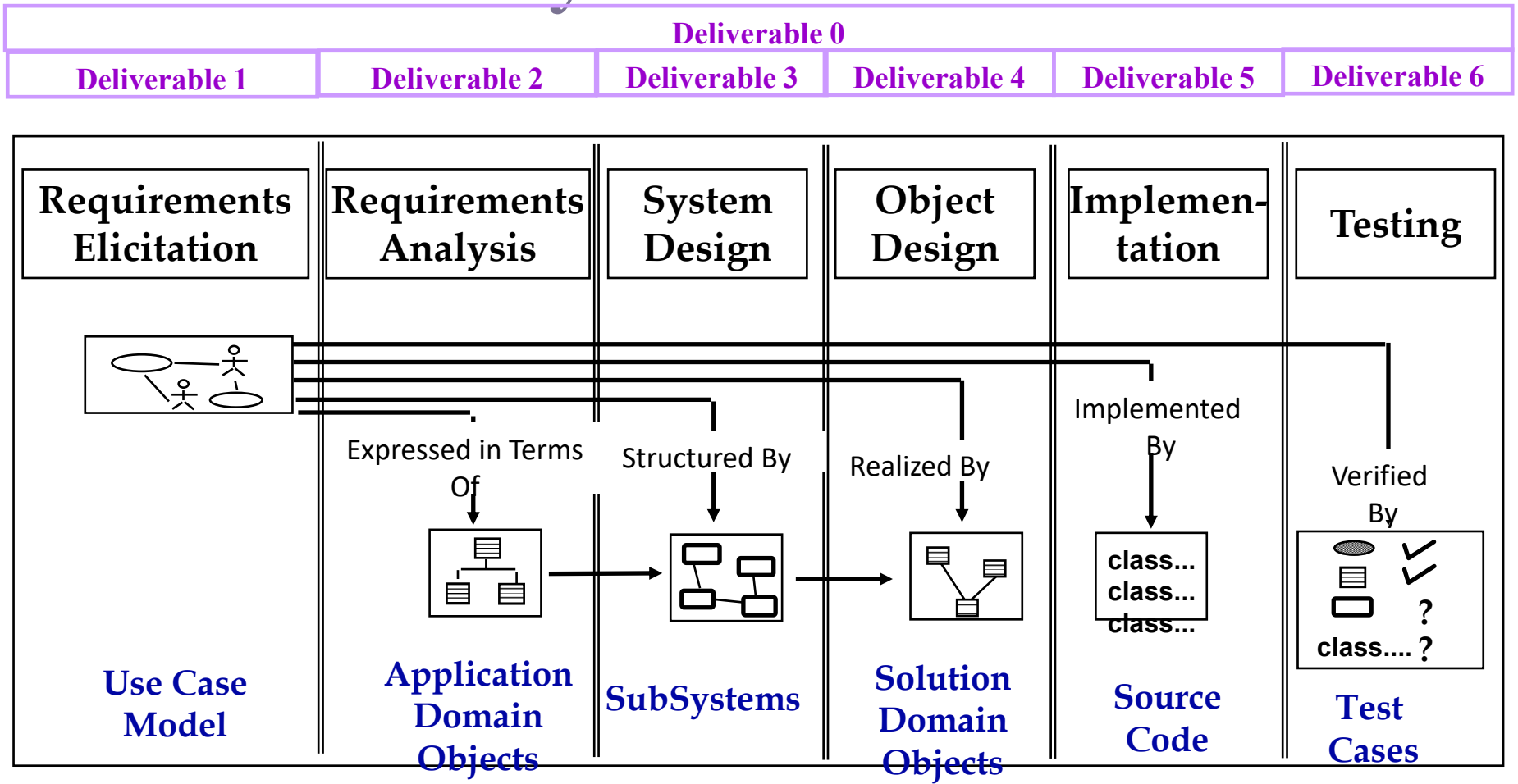
Software Methodologies Overview

Software Lifecycle Definition

- Software lifecycle:
  - Set of activities and their relationships to each other to support the development of a software system

- Typical Lifecycle questions:
  - Which activities should I select for the software project?
  - What are the dependencies between activities?
  - How should I schedule the activities?
  - Time and cost?

Software Lifecycle Activities



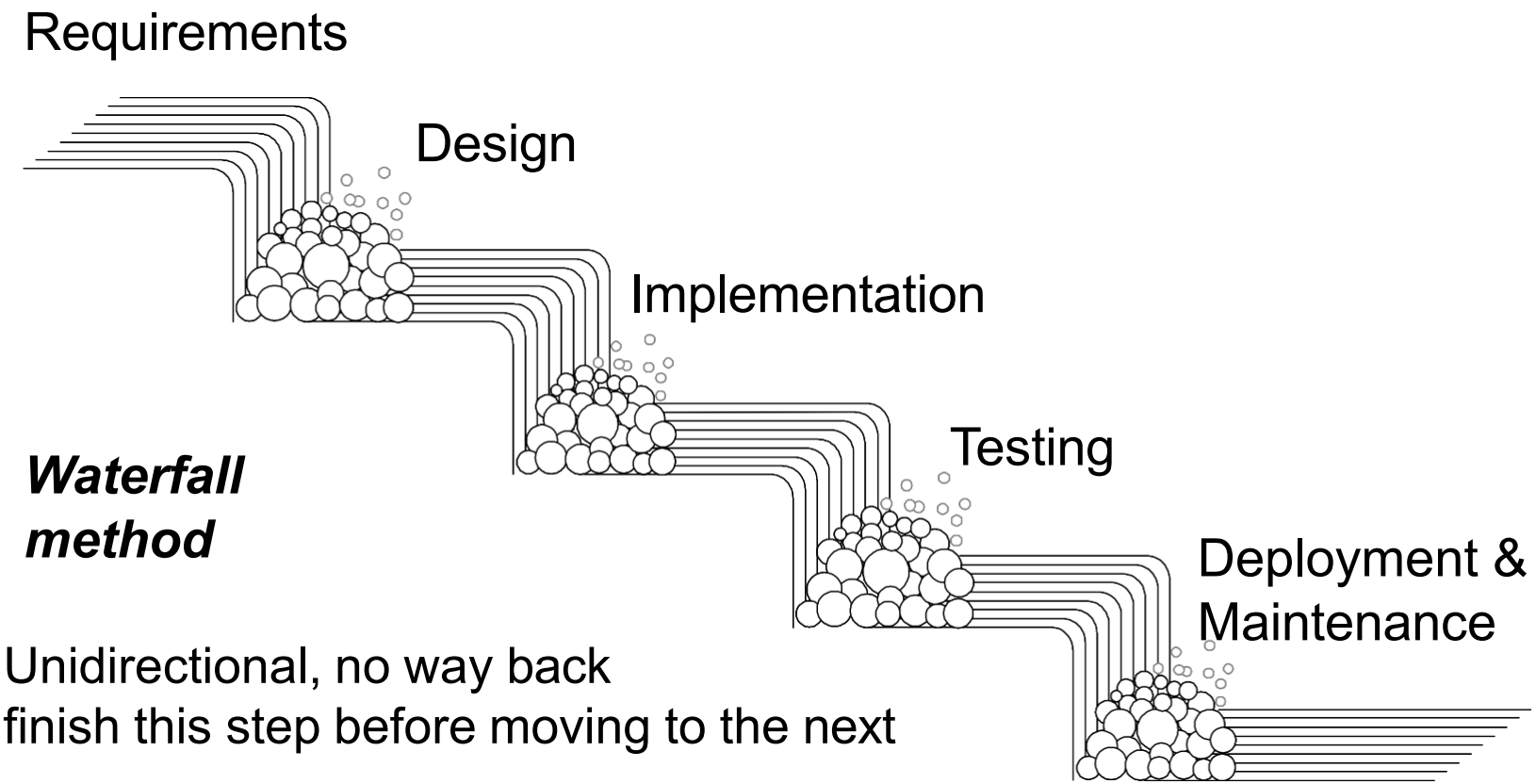
Software development methodologies

- is a framework that can be used to plan, structure and control the process of developing a software system.
- it may have several phases which may run in sequence or in cyclic/iterative way
- Different roles/actors may contribute at different phases
- Several deliverables or artifacts (e.g., documentation, software) may be produced at different phases and how to manage/maintain them
- Guidelines for communication between team or stakeholders

Software Development Methodologies

- Many examples
  - **Waterfall:** traditional approach to build software
  - **Spiral:** iterative approach to build software
  - **Rational Unified Process** (RUP) is an incremental and iterative approach
  - Other Agile methods
    - **SCRUM**
    - **eXtreme Programming** (XP)
  - There are many others e.g., **DevOps**

Traditional approach



Waterfall method

- Unidirectional, no way back
  finish this step before moving to the next

Traditional approach cont.

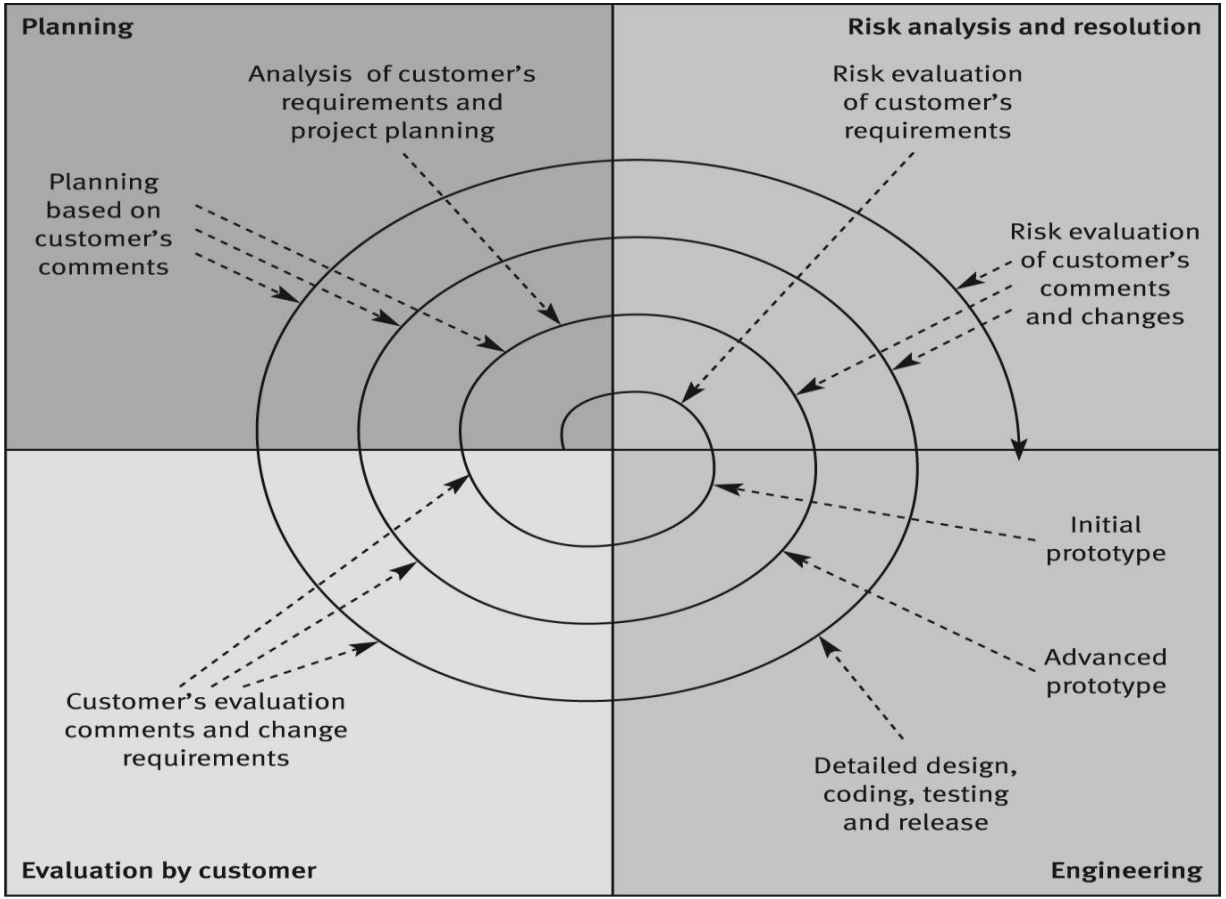
- Advantages:
  - Very logical and well organized
  - Specialized professionals in each domain
  - Tracking each step quite easier
  - If everything goes well, splendid!

- However, most things do not go as planned every time

Traditional approach cont.

- Developers skills more important than attitudes
- Customer almost out-of-the development loop i.e., customers do not have a voice in the software development process
- They consume desired software only when overall development processes have been completed
- No way of receiving continuous feedback (from customer/stakeholder)
- Monolithic Application- shipped as one whole application. Single application, single point of failure. If one part fails, whole application fails. Difficult to add new features, once the application is in production environment
- Increased Downtime

The Spiral Model



Loops: Phases of the software

Radius: expenses or costs so far

https://www.geeksforgeeks.org/software-engineering-spiral-model/

Image source: Unknown

The Spiral Model cont.

- The Spiral Model is an iterative development method
- A typical iteration between **6 months and 2 years**

  - planning, requirements analysis, risk analysis, design and architecture, and then a release of a prototype, customer feedback

  - these steps are repeated until the project is either ended or finished
- Role of project manager is important to define # of iterations (varies from one project to another)

+ve: Better risk management, good for high risk projects

-ve:

- Large number of intermediate stages requires excessive documentation.
- End of project may not be known
- Much more complex than other SDLC models: complex process, spirals may go indefinitely

Challenges of using Waterfall or Spiral Models

- Waterfall and spiral are heavy top-down approaches
  - Inflexible structure of product
  - Well-defined sequence of independent development phases

- Many problems with these approaches
  - A lot of waiting time for developers
  - Tons of documentation
  - Can result in costly unnoticed errors and buggy code
  - Hard to incorporate new or changing customer requirements

Why don’t we think an alternative approach?

- Iterative software development process
- Keeping your customer in-the-loop i.e., customers have a voice in the software development process
- They must continually consume your software, provide feedback, and witness the results of that feedback

As an example…

- Let's imagine that we want to create a blog engine; considering the following method:
  - Create the blog display page; deliver it to the customer
  - Create the user management and membership feature; deliver it to our customer
  - Add commenting capability and management; deliver it to the customer
  - So on and so forth…
- It is a simple approach, but the customer sees the real progress of his software and gives you immediate feedback on each new feature
- It may be perfect or require tweaking, but you can quickly respond to changes: a win-win situation

Texts source: net.tutsplus.com

Agile Software Development

- Agile SD is a way of thinking about project management
- Agile is rapid deployment of an application in an organised way
- Based on iterative and incremental development
- Requirements as well as solutions evolve together
- Collaboration between cross-functional, self-organising teams
- Teams kept small (5-9 people)

UWE Bristol University of the West of England

Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

http://agilemanifesto.org/

Principles of agile manifesto

- The agile manifesto has 12 main principles:

1. Customer satisfaction
2. Adapt to changing requirements
3. Deliver frequently
4. Work together frequently
5. Build projects with motivated individuals
6. Use Face to Face communication
7. Measure progress with working software
8. Maintain a constant pace
9. Pay attention to industrial progress
10. Simplicity is essential
11. Self organize
12. Reflect and adjust

Five main principles

- The 12 main principles can be condensed to the following five:

1. Deliver Early and Often to Satisfy Customer
2. Welcome Changing Requirements
3. Face to Face Communication is Best
4. Measure Progress against Working Software
5. Simplicity is Essential

Reflect and adjust

Customer satisfaction

- Focus on Customer Collaboration
- Easy to add new features
- Late requirements changes are welcomed
- Each team actually contains a customer representative

Progress measure

- Primary measure of progress is in terms of working software, not lines of code
  - Agile projects measure progress by *the amount of software that is currently meeting customer needs*
  - They are 30% done when 30% of required functionality is working AND deployed
  - Progress is not measured in terms of phases or creating documents
  - Keeps on top of customer satisfaction and allows for more realistic and updated estimation of costs

Keep it Simple

UWE Bristol University of the West of England

- This refers to the art of maximizing the amount of work NOT done
- Agile projects always take the simplest path consistent with their current goals
- They do not try to anticipate tomorrow's problems; they only solve today's problems
- High-quality work today should provide a simple and flexible system that will be easy to change tomorrow if the need arises
- Only do the job the client wants, without any additional functionalities and improvements, your work load will lighten, and you'll achieve your goals. Ultimately, that is all the customer cares about

That’s a bit Abstract!

- The agile ethos is a bit abstract from the process of actually applying the principles in a sound methodology

- Different people interpret the manifesto in different ways

Agile Approach Overview

1st Iteration 2nd Iteration 3rd Iteration

Agile Methods

UWE Bristol University of the West of England

- The popular agile methods

  Scrum, Extreme Programming (XP), Crystal Clear, and DSDM (Dynamic Systems Development Methods)

- The other methods available are Agile Unified Process (AUP), Agile Modeling, Adaptive Software Development, FDD (Feature Driven Development), and Lean Software Development
- Self-study: Open source project management tools for agile teams: https://opensource.com/article/18/2/agile-project-management-tools
- * Details on Agile and other Methodologies will be covered in the next lecture

Summary

- Overview of software methodologies
- Software engineering is a problem solving activity
  - Developing quality software for a complex problem within a limited time while things are changing
- There are many ways to deal with complexity
  - Modeling, decomposition, abstraction, hierarchy
- We'll look analysis and design activities in more detail from next week
- **Next Lecture**
  - Methodologies continue….

Reference

- Chapter 1, Bernd Bruegge, Allen Dutoit: "Object-Oriented Software Engineering: Using UML, Patterns, and Java", Prentice Hall

- More on Software Development Models https://www.geeksforgeeks.org/software-engineering-classical-waterfall-model/?ref=lbp